

AI Programming Foundations Project

Module Summary

Maintenance Work Order Data Workflow

Student: Peyman Mohammad Hassan

Dataset: Synthetic CMMS Maintenance Work Orders

Repository: [ai-programming-foundations-project](#)

Submission Date: September 2026

This report documents the data workflow submitted for the AI Programming Foundations capstone module. The project uses only synthetic academic data and contains no real customer, employee, technician, property, or asset records.

Overview

This project builds a reproducible Python data workflow using a synthetic computerized maintenance management system (CMMS)-style work-order dataset. The workflow covers data ingestion, cleaning, exploratory data analysis (EDA), visualization, interpretation, and reproducibility. The dataset is designed for academic use and contains no real customer, employee, technician, property, or asset records. The Project 1 submission repository is available at <https://github.com/peymangraph/ai-programming-foundations-project>.

Dataset Description

The dataset represents simulated maintenance work orders similar to records that could appear in a facilities or asset-management system. The generator creates 8,000 base work orders and intentionally injects duplicates and data-quality defects, producing 8,040 raw rows before cleaning. After duplicate removal and cleaning, the workflow retains 8,000 work orders with 18 columns. Key variables include asset type, asset age, asset condition, asset criticality, maintenance type, issue type, previous failures in the last 12 months, days since last service, preventive-maintenance overdue days, occupancy impact, safety-related status, estimated repair cost, resolution hours, and work-order priority. The synthetic dataset is generated from project code using a fixed seed so that the data can be recreated without exposing sensitive operational records.

Workflow Description

The workflow proceeds through five stages. First, the notebook ingests the synthetic CSV with Pandas, displays the initial rows, and confirms dataset dimensions. Second, reusable cleaning functions standardize categorical labels, remove duplicate work-order IDs, identify documented invalid values, and impute selected missing fields. Third, an EDA function summarizes priority frequencies, asset categories, and numerical characteristics. Fourth,

Table 1: Selected variables and their analytical roles.

Variable	Role in the workflow
<code>asset_type</code>	Groups maintenance records by equipment or building-system category.
<code>asset_condition</code>	Describes simulated condition and is checked for missing values.
<code>previous_failures_12m</code>	Supports EDA of maintenance history across priority levels.
<code>estimated_repair_cost</code>	Supports cost summaries and asset-type comparisons.
<code>resolution_hours</code>	Used descriptively; it is recognized as a post-completion variable.
<code>priority</code>	Main descriptive outcome with Low, Medium, High, and Emergency classes.

the notebook creates four labeled visualizations. Finally, a written summary interprets the observed patterns, states assumptions and limitations, and identifies how the workflow could support later machine-learning work.

This structure reflects the reproducible-workflow principle that analytical steps should be explicit, inspectable, and rerunnable rather than dependent on undocumented manual edits. Danchev (2022) emphasizes reproducible data-science practices that connect code, computational environments, data, and documented analytical decisions. That principle motivated the fixed random seed, reusable functions, version-controlled source code, and dependency documentation used in this project.

Key Decisions and Assumptions

Several cleaning choices were made specifically for the synthetic dataset. Variations in capitalization and whitespace are standardized so equivalent labels are not treated as separate categories. Duplicate work-order IDs are removed because retaining them would double-count maintenance events and distort summary statistics. Negative asset ages and deliberately injected extreme values are treated as invalid according to the known rules of the generator. Missing asset condition is represented as “Unknown” rather than assigning an unsupported condition, while selected numerical missing values are imputed using group

medians with an overall median fallback.

These choices are intentionally documented because data cleaning can materially change analytical results. Van den Broeck et al. (2005) describe data cleaning as a structured process of detecting, diagnosing, and addressing data abnormalities rather than an invisible preprocessing step. The project therefore preserves the raw data, makes cleaning rules visible in reusable functions, and treats those rules as dataset-specific assumptions rather than universal standards.

The visualizations were also chosen to answer distinct descriptive questions. Figure 1 shows the class distribution of work-order priority. Figure 2 compares previous-failure history across priority groups. Figure 3 compares median estimated repair cost across asset types. Figure 4 compares resolution-time distributions across priority classes. Each plot includes a title and labeled axes as required by the project rubric.

Results and Interpretation

The cleaning workflow converts a deliberately imperfect raw dataset into a consistent analysis-ready dataset while retaining the original raw source. After cleaning, the dataset contains 8,000 unique work orders. The priority distribution contains 1,756 Low, 3,693 Medium, 1,997 High, and 554 Emergency work orders. Figure 1 therefore shows that Medium is the largest class and Emergency is the smallest.

Figure 2 shows that average previous-failure counts differ across priority groups, illustrating how maintenance history may be associated with work-order severity in the synthetic data. Figure 3 shows that median estimated repair cost varies across asset categories, which demonstrates the value of grouped descriptive analysis for heterogeneous equipment populations. Figure 4 shows overlapping resolution-time distributions across priority levels. Because resolution time is known only after work completion, it is useful for descriptive analysis but should not be treated as an intake-time predictor in later machine-learning work.

These findings are descriptive demonstrations of the workflow rather than operational

conclusions. The generator intentionally encodes relationships among asset condition, criticality, failures, safety indicators, cost, and priority. The observed patterns therefore confirm that the analytical workflow exposes designed structure in the data; they do not establish causal relationships in real facilities.

Responsible Practice

Cleaning and data handling can introduce bias or misleading results even when the programming logic is correct. For example, deleting every row with missing information could disproportionately remove certain asset categories or priority levels if documentation quality differs systematically. Median imputation preserves rows but may reduce variability and can hide the reason values are missing. Van den Broeck et al. (2005) support the need to identify and document abnormalities and the decisions used to address them.

The workflow reduces these risks by preserving the raw dataset, documenting cleaning rules, using transparent transformations, and explicitly describing limitations. The use of synthetic data also avoids exposing real customer or employee records. However, synthetic data introduce a different limitation: the distributions and relationships reflect assumptions encoded by the generator. Any real-world use would require representative operational data, validation of category definitions and cleaning thresholds, and appropriate human and organizational review.

Reproducibility

A reviewer can reproduce the workflow by cloning the repository, creating a Python environment, installing dependencies with `pip install -r requirements.txt`, and executing `data_workflow.ipynb` from top to bottom. The dataset generator uses a fixed random seed, making the synthetic dataset deterministic. The repository also uses Git version control with multiple commits and an additional development branch beyond `main`. These practices connect code, data-generation logic, environment information, and analytical decisions

in a form that another reviewer can rerun and inspect, consistent with the reproducibility principles discussed by Danchev (2022).

The dedicated Project 1 repository is a focused submission derived from a broader maintenance-intelligence capstone project. The broader continuation, including later statistical-analysis, machine-learning, and NLP work, is maintained at <https://github.com/peymangraph/maintenance-ml-priority-prediction>. Keeping the Project 1 submission focused on the rubric while linking to the broader repository preserves both submission clarity and the authentic development history.

Limitations

The primary limitation is that all records are synthetic. The data are appropriate for demonstrating a safe and reproducible programming workflow, but the observed distributions and associations cannot be generalized to real buildings, assets, technicians, or maintenance organizations. A second limitation is that the cleaning thresholds are based on the known synthetic generator. In a production dataset, invalid-value rules would need to come from an authoritative data dictionary and domain expertise. Finally, the imputation strategy is transparent but simple; a real analysis would investigate the mechanism and operational meaning of missing data before selecting a final treatment.

Conclusion

The project satisfies the goals of an AI Programming Foundations data-workflow module by combining Python programming, reusable functions, tabular data handling, EDA, visualization, interpretation, responsible data practice, and reproducibility. The result is a compact but professional workflow that can be executed from beginning to end and inspected by another reviewer. It also provides a sound foundation for later statistical and machine-learning extensions without overstating what can be concluded from synthetic data.

References

- Danchev, V. (2022). Reproducible data science with Python: An open learning resource. *Journal of Open Source Education*, 5(56), 156. <https://doi.org/10.21105/jose.00156>
- Van den Broeck, J., Cunningham, S. A., Eeckels, R., & Herbst, K. (2005). Data cleaning: Detecting, diagnosing, and editing data abnormalities. *PLoS Medicine*, 2(10), e267. <https://doi.org/10.1371/journal.pmed.0020267>